TAB Wellness Collective — Technical Architecture

Status: Governing technical reference for the build · **Version:** 1.0 · **Date:** 2026-09-27
Sources reconciled: Step 16 §16.5–16.6 (Website & Digital Experience), Decision & Risk Register D-001 to D-053. Where this document and Step 16 differ, this document governs the build; Step 16 remains the strategy source. Changes to anything marked **Decision** need a register entry first.

---

1. Architecture principles

1. **Own the experience, rent operations** (D-001). TAB owns the website, the client relationship model, plans, programs, and progress. Scheduling (Timely) and payments (Stripe) stay with specialist systems.
2. **One application, two audiences.** A single Next.js app serves the public site, the My TAB client portal, and the practitioner workspace.
3. **Sensitive by design** (D-042). The client domain is treated as a health-data system: RLS, audit, MFA, BAAs. Nothing sensitive leaks into analytics, logs, URLs, email subjects, or notifications.
4. **Model for a team, build for one** (D-045). Roles and care-team assignment are data; the MVP ships owner-practitioner and client only.
5. **Configuration over code.** Business-editable values live in `config/`; content lives in Payload-shaped files until the CMS is wired.
6. **Managed infrastructure** (Step 16.6). Open source does not mean self-hosted. No self-hosting without a stated reason.
7. **Supported interfaces only.** No scraping, no unsupported two-way sync. Timely has no public API (D-003), so we link out.

---

2. Logical architecture

```
flowchart TB
  subgraph Visitors
```

```
  V[Public visitor]
  C[Client]
  P[Practitioner]
end
subgraph App["Next.js app on Vercel"]
  SITE[Public website<br/>app/(site)]
  PORTAL[My TAB portal<br/>app/my-tab]
  WS[Practice workspace<br/>app/practice]
  MW[Middleware<br/>redirects + auth guard]
  SA[Server actions + route handlers]
end
subgraph Data["Owned data"]
  CF[Content files → Payload CMS]
  SB[(Supabase Postgres<br/>Auth · RLS · Storage · Audit)]
end
subgraph External["Systems of record we rent"]
  T[Timely<br/>scheduling]
  S[Stripe<br/>billing]
  R[Resend<br/>transactional email]
  PL[Plausible<br/>analytics]
end
V --> MW --> SITE
C --> MW --> PORTAL
P --> MW --> WS
SITE --> CF
PORTAL --> SA --> SB
WS --> SA
SITE -. link out .-> T
PORTAL -. link out .-> T
SA --> S
S -. webhooks .-> SA
SA --> R
SITE -. public pages only .-> PL
```

Layer	Components	Status
Experience	Public website, My TAB, Practice workspace (Next.js 15, React 19, Tailwind 4)	Built; portal and workspace on demo data
Edge	`middleware.ts` : legacy 308 redirects, auth guard for portal routes	Built
Application	Server actions (forms, check-ins, billing portal), auth callback route	Built; Supabase paths partial
	Typed content files (`content/`) shaped for	

Content	Payload collections	Built; Payload not wired
Client domain	Supabase: schema, RLS, audit triggers (`supabase/migrations/0001_my_tab.sql`)	Written and syntax-checked; not applied
External	Timely, Stripe, Resend, Plausible	Integration points coded; accounts to configure
Observability	Audit log table; app monitoring, alerting, backups	Audit built; the rest not started

---

3. Technology stack

Concern	Technology	Decision	Status	Notes
Framework	Next.js 15 (App Router), React 19, TypeScript strict	D-005	Built	Server Components by default
Styling	Tailwind CSS 4 with canonical tokens	D-053	Built	`app/globals.css` mirrors the token document
Fonts	Cormorant Garamond, Manrope via `next/font`	D-053	Built	Self-hosted at build time, no runtime Google calls
CMS	Payload CMS	D-036	**Not wired**	See A-02
Database, auth, storage	Supabase (Postgres, Auth, Storage)	D-005, D-042	Schema written	HIPAA-eligible tier required before real data (A-01)
Scheduling	Timely (Elevate)	D-003	Link-out	No public API; Innovate + Zapier later (D-015)
Payments	Stripe	D-004, D-052	Billing-portal action built	Webhooks not built
Transactional		**Proposed**		Form notifications

email	Resend	**D-055**	Built	only; see A-04
Analytics	Plausible; Meta pixel only when ads start	D-046	Built (config-gated)	Supersedes Step 16's "Matomo or Plausible"
Hosting	Vercel	D-005	Not deployed	Pro plan; BAA add-on only if A-01 requires it
DNS / edge	Cloudflare	D-005	Not used	See A-03
Video delivery	Bunny Stream	D-005	Not used	See A-03
Validation	Zod	Build choice	Built	All form and action input
Accessibility	WCAG 2.2 AA	D-037	Built into components	Needs an audited pass before launch

---

4. System-of-record matrix (D-035, D-052)

Data domain	**System of record**	**App behavior**
Portal identity	Supabase Auth	Passwordless email link; MFA for staff; no self-signup
Client whole-person profile	Supabase `clients`	Authoritative, except fields deliberately mastered in Timely
Appointments, appointment payments, packages	Timely	Link out; never recreate calendar logic
Booking-coupled consult forms	Timely (where retained)	Reference only; don't duplicate
Wellness Plan	Supabase `wellness_plans`	Authoritative; clients see `status = 'shared'` only
Program templates	Payload (future) / `program_templates` (now)	Templates authored once, personalized per client

Enrollments, activity responses	Supabase	Authoritative
Check-ins, milestones, progress reviews	Supabase	Authoritative
Session summaries	Supabase `session_summaries`	Client-visible only when `shared_with_client`
Practitioner notes	Supabase `practitioner_notes`	**Never** client-visible; no client RLS policy exists
Public content (services, team, FAQ, resources)	Content files → Payload	Rendered on site and in portal
Assigned resources	Supabase `resource_assignments` → Payload id	Client-specific
Digital products, coaching subscriptions, invoices, cards	Stripe	Portal shows safe summary; card data never touches the app
Start Where You Are submissions	Email inbox (Resend)	Not persisted in the app (A-04)

Integration identifiers

One internal key per client; external IDs are mapped, never used as keys.

Field	Where	Purpose
`clients.id`	Supabase	`tab_client_id` , the permanent internal UUID
`clients.user_id`	Supabase	Link to Supabase Auth (`portal_user_id`)
`clients.timely_client_id`	Supabase	Set only after confident matching
`clients.stripe_customer_id`	Supabase	Unique; one Stripe customer per client (D-052)
`clients.primary_practitioner_id`	Supabase	Default care-team owner

`clients.last_synced_at`	Supabase	Reconciliation tracking

---

5. Integration patterns

Timely (D-003). Booking and client-login URLs come from `config/site.ts`, with optional per-service deep links. If unset, CTAs fall back to `/book`, which offers email scheduling. No data sync at launch. When Timely Innovate is adopted (D-015), use the documented Zapier feed for next-appointment summaries only.

Stripe (D-004, D-052). Timely owns appointment and package payments. Stripe direct owns digital products and coaching subscriptions. Built: `openBillingPortal()` creates a Customer Portal session for the signed-in client. To build: Checkout for sellable programs (D-044), and a webhook route (`/api/stripe/webhook`, signature-verified) that creates enrollments on purchase and updates membership status. Store only IDs and status.

Resend. `lib/email.ts` sends plain-text notifications to `info@tabwellnesscollective.com`. Subjects never carry personal or health details. Without `RESEND_API_KEY` it logs to the server console. Sender domain must be verified.

Plausible and Meta (D-046). Loaded by `components/Analytics.tsx` only when configured, and never on `/my-tab`, `/practice`, `/start`, or `/contact`. No custom events carry form content.

Legacy redirects (D-040). `config/redirects.ts` maps 64 crawled tabmassage.com URLs plus prefix rules. Middleware issues single-hop 308s; requests on the tabmassage.com host land on the canonical `www.tabwellnesscollective.com`.

Payload (D-036, future). Content files map 1:1 to collections: `services`, `pages` (globals), `faqs`, `team`, `resources`, `programTemplates`. Payload runs inside the same Next.js app on the same Postgres instance under a separate schema, or its own database; decide under A-02.

---

6. Data architecture

Schema: `supabase/migrations/0001_my_tab.sql` (18 tables). Domain types: `lib/portal/types.ts` (keep in sync).

Group	Tables
People and	

access	`practitioners`, `clients`, `care_team`
Plan	`wellness_plans` (versioned)
Programs	`program_templates`, `program_modules`, `program_activities`, `enrollments`, `activity_responses`
Progress	`check_ins`, `milestones`
Session records	`session_summaries` (shareable), `practitioner_notes` (internal only)
Library and paperwork	`resource_assignments`, `form_templates`, `form_requests`, `documents`
Governance	`audit_log`

Authorization model. RLS on every table, using `security definer` helpers with a fixed `search_path`: `current_client_id()`, `current_practitioner_id()`, `is_owner()`, `can_access_client(uuid)`. Clients read their own rows, and only shared rows where sharing applies. Practitioners reach clients through primary assignment or `care_team`. Owner access is universal but audited. Staff-only routes also check `current_practitioner_id()` in the app layer.

Audit. Triggers write insert, update, and delete events on sensitive tables to `audit_log`. Reads are not audited at the database level (see A-05).

Data access in the app. Screens call only `lib/portal/repo.ts`, which switches between demo data and `supabase-repo.ts` via `siteConfig.portal.dataSource`. Queries run as the signed-in user; the service-role key is never used for client-facing reads.

---

7. Security baseline (Step 16.6, D-042)

Control	Status
RLS so clients see only their own data; practitioners only assigned clients	Written, **untested**
Practitioner notes structurally unreachable by clients	Written (no client policy)
Staff-only workspace guard	Built (app layer + RLS)

MFA mandatory for staff	Supabase setting, to enable
Passwordless client auth, no self-signup	Built (`shouldCreateUser: false`)
Audit of sensitive writes	Written
No sensitive data in analytics, URLs, email subjects, notification previews	Built into components and actions
Security headers (nosniff, frame deny, referrer, permissions)	Built
`noindex` and `no-store` on portal routes	Built
Content-Security-Policy	**Not built** (launch task)
BAAs with Supabase and Vercel	**Gated** on A-01
Encrypted storage and backups; tested restore	Managed by Supabase; restore test not done
Environment separation (dev, preview, prod; synthetic data outside prod)	To set up
Dependency patching and vulnerability scanning	To set up (Dependabot or Renovate)
File uploads: signed URLs, type and size limits, malware scanning	Not built

---

8. Environments and deployment

Environment	Hosting	Data	Portal mode
Local	`npm run dev`	Demo data	`mock`
Preview	Vercel preview deployments	Synthetic Supabase project	`supabase`
Production (Phase 1)	Vercel	None	`enabled: false` (interim page to Timely)
Production	Vercel (+ BAA if	HIPAA-eligible	`supabase`

(Phase 2)	required)	Supabase project	

Secrets live only in Vercel environment variables (`.env.example` lists them). DNS for `tabwellnesscollective.com` and `tabmassage.com` points at Vercel at cutover (Step 19).

---

9. Open architecture decisions

ID	Question	Recommendation	Blocks
A-01	Gate HIPAA-tier spend (≈$1,300/month; R-025) on legal review?	Yes. Keep the sensitive-data design; buy the tier only when counsel confirms and Phase 2 data features go live	Phase 2
A-02	Wire Payload before or after Phase 1 launch?	After. Launch on content files; wire Payload when a non-developer needs to edit, or when program templates move to the CMS	Content editing
A-03	Keep Cloudflare and Bunny Stream from D-005?	Cloudflare: drop for now; Vercel DNS and edge cover Phase 1, revisit only for a specific need. Bunny Stream: defer to Phase 2 and decide when program video exists; Supabase Storage or unlisted hosting covers early volume	Amends D-005
A-04	Start form: email only, or persist submissions?	Email only for Phase 1, with the "no medical details" guidance. When My TAB launches, route intake into the portal instead of the inbox	D-042 boundary
A-05	Audit sensitive reads, not just writes?	Yes for practitioner notes and plans in Phase 2, via an audited read function; confirm with counsel	Phase 2
A-06	Monitoring and error tracking	Vercel logs plus Sentry with PII scrubbing; never capture request bodies from portal routes	Launch

---

10. Reconciliation with earlier sources

Item	Step 16	Register	This document

CMS	Payload preferred	D-036 Payload	Payload, not yet wired (A-02)
Analytics	Matomo or Plausible, optional GA4	D-046 Plausible; pixels only with ads	Plausible (Step 16 superseded)
Cloudflare, Bunny Stream	Not mentioned	D-005 includes both	Open (A-03)
Transactional email	Not specified	None	Resend (proposed D-055)
Health data in owned systems	Sensitive-data portal	D-042 supersedes D-002	Designed for it; spend gated (A-01)
Timely integration	Link-out; Zapier on Innovate	D-003, D-015	Link-out built
Architecture Plan (2026-09-25)	—	Retired by D-041	Not used